



PRINCÍPIOS DE

# Computação Paralela e Distribuída

*Do processador único aos sistemas em nuvem*

**Fonte:** Buyya, R. et al. — Mastering Cloud Computing, 2ª ed.  
(Cap. 2)

# APRESENTAÇÃO

**Sistema Paralela:** várias tarefas ao mesmo tempo, memória compartilhada, numa mesma máquina (ex.: GPU renderizando vídeo, VM com vários núcleos).

**Sistema Distribuída:** máquinas independentes, sem memória em comum, colaborando pela rede (ex.: SETI@home, data centers da AWS/Google). A nuvem usa os dois juntos.

# Objetivos de aprendizagem

1

Diferenciar computação paralela de computação distribuída, com exemplos reais

2

Reconhecer arquiteturas SIMD e MIMD em hardware do dia a dia (GPU vs. CPU)

3

Explicar por que mais processadores não geram desempenho proporcional

4

Identificar estilos arquiteturais reais: cliente/servidor e peer-to-peer

5

Comparar RPC, objetos distribuídos e Web services (SOAP/REST)

## SEÇÃO 2.1

# Da era sequencial à era paralela

### CENÁRIO REAL

- ✦ ENIAC (1945): executa instruções estritamente uma após a outra
- ✦ Cray-1 (1976): aplica a mesma operação a vários dados ao mesmo tempo

 *O que a GPU do seu computador tem em comum com o Cray-1 de 1976?*

### CONCEITO TÉCNICO

- ✦ Ambas nasceram da mesma ideia: usar múltiplos processadores trabalhando ao mesmo tempo, aplicando a mesma operação a vários dados simultaneamente, em vez de processar tudo em sequência.

## SEÇÃO 2.1

# Da era sequencial à era paralela

O que muda entre os dois é a escala e o contexto de uso:

o Cray-1 era uma máquina de milhões de dólares usada por laboratórios e centros de pesquisa (fase de P&D/comercialização inicial);

a GPU é a mesma ideia arquitetural, décadas depois, comoditizada a ponto de vir embutida em qualquer notebook comum — exatamente o padrão de amadurecimento tecnológico (P&D → comercialização → comoditização).



### CONCEITO TÉCNICO

- ✦ Toda tecnologia passa por 3 fases: P&D → comercialização → comoditização
- ✦ É exatamente o caminho que a própria computação em nuvem também percorreu

# Computação paralela vs. distribuída

### CENÁRIO REAL

- ✦ Seu notebook: vários núcleos compartilham a mesma memória RAM
- ✦ Cliente/Server: milhares de PCs distintos, sem memória em comum

### CONCEITO TÉCNICO

- ✦ Paralelo: fortemente acoplado, memória compartilhada, geralmente homogêneo
- ✦ Distribuído: termo mais amplo — nós heterogêneos e fisicamente dispersos

# Taxonomia de Flynn: SIMD vs. MIMD

### GPU — arquitetura SIMD

- ✦ Milhares de núcleos executam a mesma instrução
- ✦ Cada núcleo opera sobre um dado diferente
- ✦ Ex.: renderizar pixels de uma imagem

### CPU multicore — arquitetura MIMD

- ✦ Poucos núcleos, cada um independente
- ✦ Cada núcleo roda instruções e dados diferentes
- ✦ Ex.: navegador e player de música ao mesmo tempo

Provedores de nuvem oferecem dois tipos de instância conforme a carga: **GPU (SIMD)** para tarefas que repetem a mesma operação em massa — IA, renderização, vídeo, simulações — pois CPUs comuns seriam muito mais lentas e caras nesse caso; e **CPU multicore (MIMD)** para o uso geral e flexível — servidores web, bancos de dados, microsserviços — que sustenta a maior parte da infraestrutura de IaaS.

# MIMD: memória compartilhada vs. distribuída

### CENÁRIO REAL

- ✦ Vários núcleos de uma CPU acessam a mesma RAM diretamente
- ✦ Num cluster, cada nó/servidor tem sua própria memória local

 *Se um núcleo falhar, o que acontece com o restante — em cada modelo?*

### CONCEITO TÉCNICO

- ✦ Memória compartilhada: mais simples, porém menos tolerante a falhas (se falha, derruba tudo)
- ✦ Memória distribuída: mais escalável e tolerante — hoje é o modelo dominante



# Granularidade e abordagens de paralelismo

"de que tamanho" você corta um problema para paralelizar, e "de quantas formas" você pode distribuir esse trabalho entre processadores.

**Granularidade (o tamanho do pedaço):** quanto maior o "pedaço" de código paralelizado, mais grosso é o grão; quanto menor, mais fino.

- **Grão grosso:** um programador divide manualmente o app inteiro em processos separados (ex.: um processo cuida do banco, outro da interface).
- **Grão fino:** o compilador sozinho identifica um laço de repetição e paraleliza cada iteração automaticamente, sem intervenção humana.
- Entre esses extremos existem os graus médio e muito fino, do processo até a instrução individual.

# Granularidade e abordagens de paralelismo

### CENÁRIO REAL

- ✦ Um compilador paraleliza sozinho um laço de repetição (grão fino)
- ✦ Um programador divide manualmente um app em processos (grão grosso)

### CONCEITO TÉCNICO

- ✦ Grãos: grande → médio → fino → muito fino (do processo à instrução)

# Granularidade e abordagens de paralelismo

### Abordagens (como você distribui o trabalho):

- **Paralelismo de dados:** divide os dados em pedaços e aplica a mesma operação a cada pedaço.
- **Paralelismo de processos:** divide o problema em atividades diferentes, cada uma rodando em paralelo.
- **Farmer-and-worker:** um processador "mestre" distribui tarefas para "trabalhadores" e depois junta os resultados.

Saber calibrar o grão é o que separa um programa paralelo bem otimizado de um que gasta mais tempo se coordenando do que processando.

# Leis de cautela: mais núcleos ≠ mais velocidade

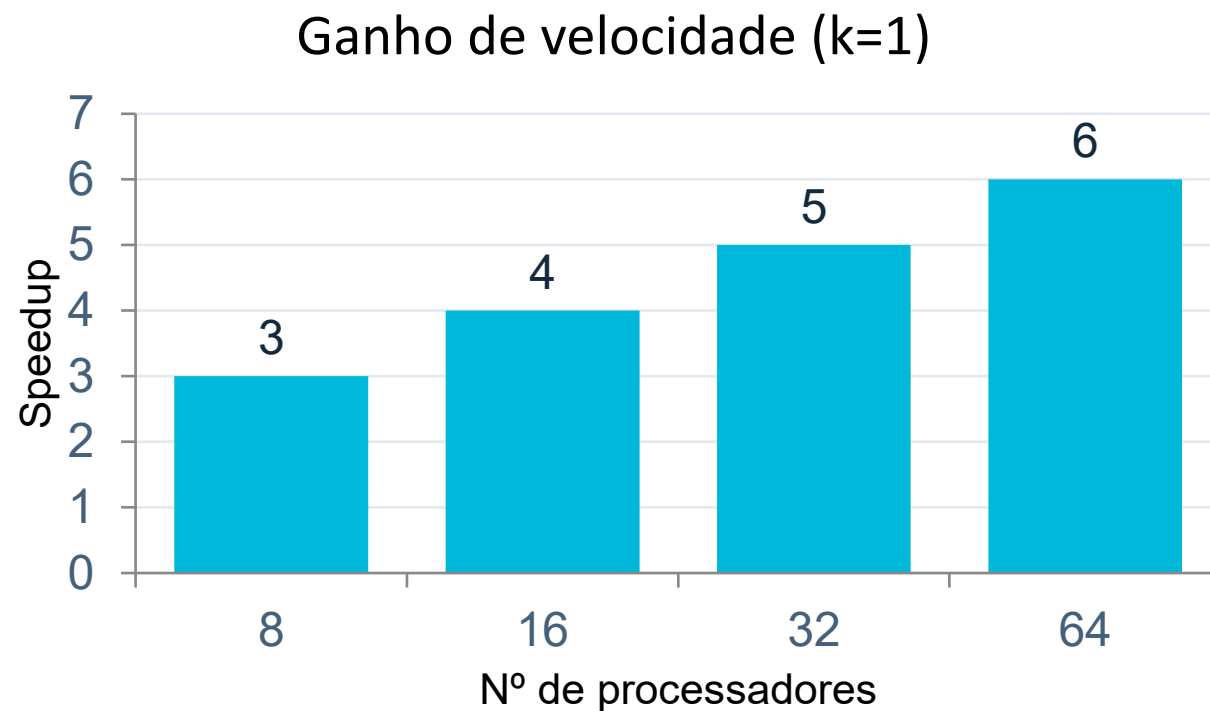


### CONCEITO TÉCNICO

Velocidade cresce apenas com

$$y = k \cdot \log(N)$$

— não proporcionalmente ao nº de processadores (N), por causa da sobrecarga de comunicação.



Dobrar o número de processadores não dobra a velocidade — cada vez que você multiplica os processadores, o ganho de desempenho cresce cada vez menos, porque parte do tempo passa a ser gasto fazendo eles se comunicarem entre si em vez de trabalhar.

# Sistema distribuído: da definição às camadas da nuvem



### CENÁRIO REAL

- ✦ Você usa um app na nuvem pelo navegador — a camada SaaS
- ✦ Por baixo, frameworks (PaaS) rodam sobre VMs alugadas (IaaS)

Nuvem em três camadas empilhadas:

**IaaS** = só o hardware/VM crua (você cuida de tudo);

**PaaS** = infraestrutura + plataforma pronta para programar (você só cuida do código);

**SaaS** = aplicativo pronto, o usuário final só usa pelo navegador (Gmail, Netflix). Cada camada roda em cima da anterior, e o usuário comum normalmente só vê o SaaS, no topo.

# Sistema distribuído: da definição às camadas da nuvem



### CONCEITO TÉCNICO

- ✦ Sistema distribuído = vários computadores que parecem um só ao usuário
- ✦ Camadas: hardware/SO → middleware → aplicações (= IaaS → PaaS → SaaS)



**FERRAMENTA:** O console da AWS/Azure/GCP mostra as VMs (IaaS) por trás de qualquer serviço na nuvem

**Sistema distribuído:** vários computadores separados que, para o usuário, parecem um só. Ele é organizado em camadas empilhadas — hardware/sistema operacional, depois middleware, depois as aplicações — que correspondem exatamente a IaaS → PaaS → SaaS. E dá pra ver isso na prática: o console da AWS/Azure/GCP mostra as VMs (a camada IaaS) que sustentam por baixo qualquer serviço de nuvem.

## SEÇÃO 2.4

# Estilos de sistema: cliente/servidor vs. peer-to-peer

### ▶ Cliente/servidor

- ◆ Um servidor central responde a vários clientes
- ◆ Ex.: seu navegador pedindo uma página Web
- ◆ Escala limitada: pode virar gargalo

### ◆ Peer-to-peer (P2P)

- ◆ Cada nó é cliente e servidor ao mesmo tempo
- ◆ Ex.: BitTorrent, Skype
- ◆ Mais escalável, porém mais complexo de coordenar

⚙ **Cliente/servidor centraliza a carga; P2P a distribui entre os próprios usuários.**

**Cliente/servidor:** Gmail, Netflix, bancos de dados, e-mail (um servidor central atende vários clientes).

**P2P:** BitTorrent, Skype, Bitcoin (cada nó é cliente e servidor ao mesmo tempo, sem um centro único).

# Estilos de sistema: cliente/servidor vs. peer-to-peer

**Cliente/servidor:** controle centralizado, fácil de gerenciar, mas pode virar gargalo/ponto único de falha.

**Peer-to-peer:** sem ponto central, mais resiliente e escalável, mas mais difícil de coordenar.

No paralelo/distribuído, a escolha entre eles é um trade-off (quando você ganha algo de um lado e perde algo do outro) entre simplicidade centralizada e resiliência distribuída.



# Comunicação entre processos: pipes e pub/sub



### CENÁRIO REAL

- ✦ No terminal: ``cat arquivo | grep termo | wc -l``
- ✦ Cada comando processa e repassa o resultado ao próximo



### CONCEITO TÉCNICO

- ✦ Pipe-and-filter: dados fluem entre "filtros" que não se conhecem
- ✦ Publicação-assinatura: o publicador não conhece os assinantes (ex.: push)



**FERRAMENTA:** Abra um terminal e encadeie comandos reais com ``|`` para observar o pipe-and-filter

# Comunicação entre processos: pipes e pub/sub



### CENÁRIO REAL

- ✦ No terminal: ``cat arquivo | grep termo | wc -l``
- ✦ Cada comando processa e repassa o resultado ao próximo



### CONCEITO TÉCNICO

- ✦ Pipe-and-filter: dados fluem entre "filtros" que não se conhecem
- ✦ Publicação-assinatura: o publicador não conhece os assinantes (ex.: push)



**FERRAMENTA:** Abra um terminal e encadeie comandos reais com ``|`` para observar o pipe-and-filter

# Comunicação entre processos: pipes e pub/sub

**Pipe-and-filter:** comandos encadeados com |, cada um processa e passa adiante sem conhecer os outros (ex.: `cat | grep | wc -l`).

**Pub/sub:** quem publica não sabe quem vai receber — só entrega a quem estiver "inscrito" (ex.: notificação push, inscrição no YouTube).

Alguém manda mensagem no grupo (publica) sem saber quem está online agora; cada membro do grupo (inscrito) recebe a mensagem automaticamente quando abrir o app.

São importantes porque permitem que as partes de um sistema paralelo/distribuído troquem dados sem ficarem "amarradas" umas às outras — cada uma pode ser trocada, adicionada ou rodar separada, o que torna o sistema mais fácil de escalar e manter.

# Chamada de procedimento remoto (RPC)



### CENÁRIO REAL

- ✦ Um app cliente invoca uma função que roda em outro servidor
- ✦ O cliente fica bloqueado (síncrono) até a resposta voltar



### CONCEITO TÉCNICO

- ✦ RPC estende a chamada de função além dos limites de um processo
- ✦ Parâmetros são empacotados (marshaling) para trafegar pela rede

# Chamada de procedimento remoto (RPC)

RPC é chamar uma função que roda em outra máquina, mas parece uma chamada de função normal para quem programa.

**Exemplo:** o site de uma loja chama "calcularFrete()", que na verdade executa no servidor da transportadora. O cliente fica esperando (síncrono) até a resposta voltar, e os dados precisam ser "empacotados" para viajar pela rede.

RPC permite que partes de um sistema distribuído, rodando em máquinas diferentes, se comuniquem como se fossem um programa só — sem o programador lidar com rede manualmente.

É a base de microsserviços e frameworks como gRPC, RMI, CORBA.

# Frameworks de objetos distribuídos

### CORBA

- ✦ Multiplataforma e multilinguagem
- ✦ Complexo de implementar

### Java RMI

- ✦ Usa um registro RMI para localizar objetos
- ✦ Restrito ao ecossistema Java

### .NET Remoting

- ✦ Arquitetura customizável
- ✦ Não usa registro externo



**Todos dão a ilusão de invocar um objeto local — o proxy/skeleton esconde a rede.**

**CORBA:** multiplataforma/multilinguagem, mas complexo de configurar. **Java RMI:** só entre sistemas Java, usa um registro para localizar objetos remotos. **.NET Remoting:** mundo Microsoft, arquitetura customizável, sem registro externo.

Todos fazem parecer que você chama um objeto local, escondendo a rede por trás de um proxy/skeleton.

# Web services: SOAP vs. REST

### REST

- ✦ Usa métodos HTTP: GET, POST, PUT, DELETE
- ✦ Resposta leve, normalmente em JSON
- ✦ Ex.: APIs do Twitter/X, Yahoo!, Amazon.com

### SOAP

- ✦ Mensagem XML com envelope, cabeçalho e corpo
- ✦ Mais robusto, porém mais verboso
- ✦ Descrito por um documento WSDL

 **Ambos implementam SOA: sistemas organizados como serviços reutilizáveis e interoperáveis.**

SOA é organizar um sistema como vários serviços independentes e reutilizáveis que se comunicam pela rede (ex.: serviço de pagamento, frete e estoque num e-commerce). Cada um pode ser atualizado ou substituído sem afetar os outros.

## SEÇÃO 2.5

# Web services: SOAP vs. REST

**REST:** leve, usa verbos HTTP (GET/POST/PUT/DELETE), resposta em JSON — ex.: APIs do Twitter/X, Amazon, o curl do laboratório.

**SOAP:** mais robusto e formal, mensagens em XML com envelope/cabeçalho/corpo, descrito por um WSDL — usado em sistemas corporativos que exigem mais rigor.

**Ambos implementam SOA:** dividir um sistema em serviços reutilizáveis que conversam entre si.



## SÍNTESE

### Do processador único ao ecossistema de serviços



Dos limites físicos de um único processador

→ à necessidade de coordenar múltiplas máquinas

→ até a camada de serviços que torna essa coordenação usável, em escala, pela Internet.